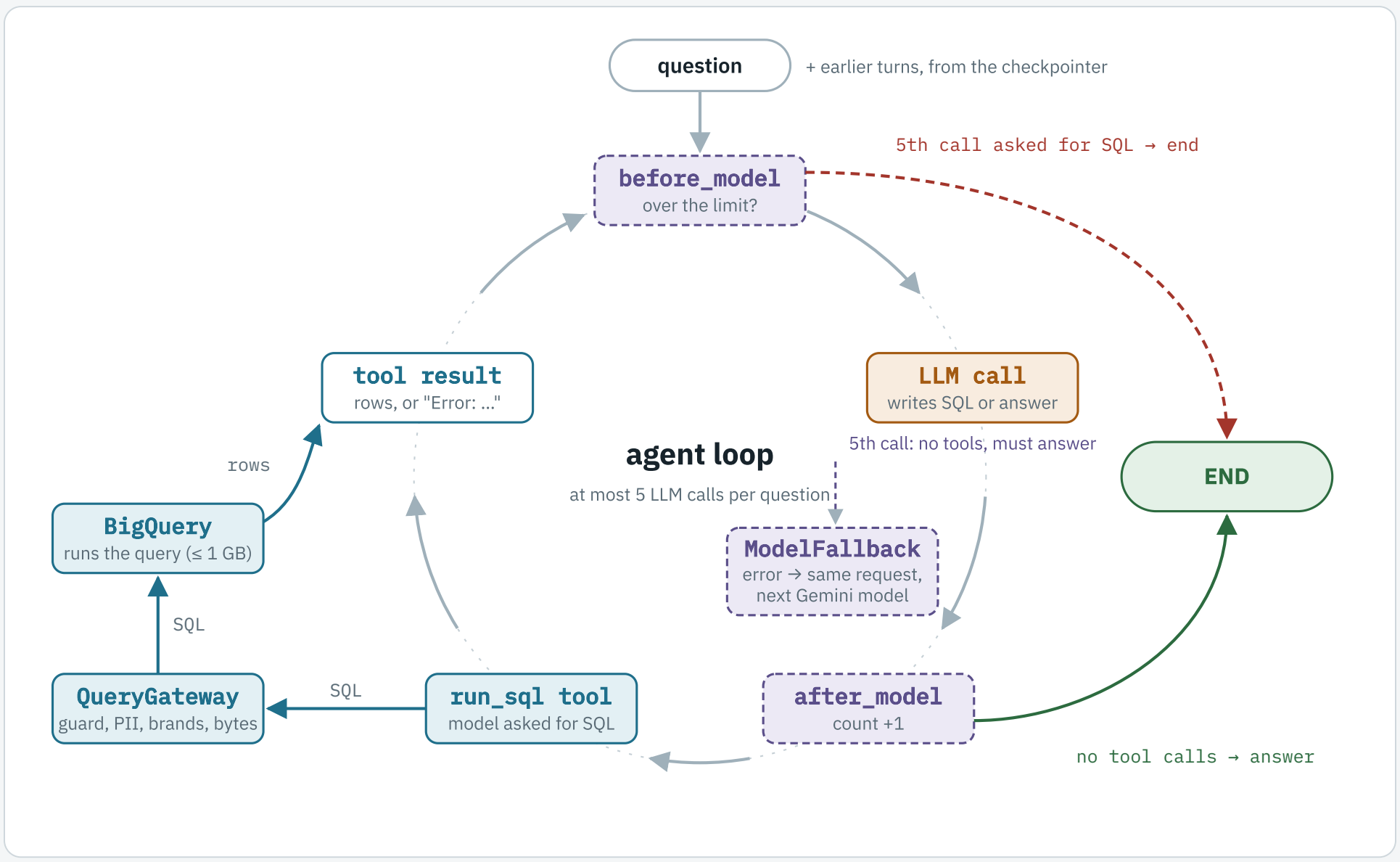
The agent loop, as a circle

Each question goes round this circle until the model answers. The database work happens outside the circle: the tool hands SQL to the gateway, the gateway to BigQuery, and the result comes back in.

middleware (CallBudget, ModelFallback) · LLM · tool and data path · end of the question



1. **`before_model`**: CallBudget checks the count. Under 5, the loop goes on. If the 5th call still asked for SQL, it **`jumps to END`** with "I couldn't finish this question...". That is only a fallback: normally the 5th call answers.
2. **`LLM call`**: the model either writes SQL (a tool call) or gives the final answer. It sees the earlier turns of the conversation too, so follow-up questions work. On the **`5th call`** CallBudget removes the tools and adds a note: answer now from the results you have, and say what is missing. If the call fails (out of quota, busy, timed out), **`ModelFallback`** sends the same request to the next Gemini model, which gets the same tools and the same last-call rule. The switch doesn't count as an extra call; see `docs/model-switching.html`.
3. **`after_model`**: CallBudget adds 1 to the count. With no tool calls, the answer goes to **`END`**. With tool calls, the loop goes on.
4. **`run_sql tool`**: hands the SQL out of the circle to the **`QueryGateway`**. The guard checks it is one SELECT over the 4 tables, the personal-data check lets it read only the columns in `schema.yaml`, age, city and other demographic columns may appear only in totals of 5+ customers, customer ids are swapped for pseudonyms such as `cust_72ac84091163`, the brand scope limits every table to the user's brands, a dry-run checks it stays under the byte limit, and only then does it run in **`BigQuery`**.
5. **`tool result`**: the rows come back into the circle as a table (with a note if rows were cut off or none matched), or `Error: …` if the gateway or BigQuery rejected the query, and the loop returns to **`before_model`**. When one reply holds several `run_sql` calls, they run at the same time and all their results come back together.
6. **`save_report tool`** (step 5, not drawn): for a report, the model writes the finished report into `save_report` instead of a plain answer. The code saves it with the owner, conversation, question and SQL, and its result goes straight to **`END`** as the answer (`return_direct`), so saving costs no extra LLM call. A typical report: 1 call that sends all its queries at once, 1 call that writes and saves it. On the 5th call there are no tools, so the model writes the report and says it can't be saved then.
7. **`list_reports tool`** (step 5, not drawn): asked "show my reports" or "which reports mention Client X?", the model calls `list_reports`. The code lists only the asking user's reports, and that list goes straight to **`END`** as the answer, so it costs 1 LLM call.
8. **`delete_reports tool`** (step 5, not drawn): asked "delete all reports mentioning Client X", the model only proposes, by calling `delete_reports`. The code finds the asking user's matching reports and **`pauses the turn`** with LangGraph's `interrupt`, carrying the exact list. The CLI shows it, and `App.confirm` resumes the turn with what the user typed: only `yes` deletes those reports, anything else cancels. The outcome goes straight to **`END`** as the answer: 1 LLM call in total.

Code: `agent.py` (CallBudget, ModelFallback, the loop via `create_agent`), `tools.py` (run_sql, save_report, list_reports, delete_reports), `reports.py` (the report store), `query_safety/gateway.py`, `query_safety/step1_guard.py`, `query_safety/step2_personal_data/columns.py`, `query_safety/step2_personal_data/small_groups.py`, `query_safety/step2_personal_data/customer_pseudonyms.py`, `query_safety/step3_brand_scope/brand_scope.py`, `db/bigquery.py`; the conversation is kept in `app.py`, which also resumes a paused turn (`App.confirm`). See also `docs/sql-journey.html` (what the gateway does to the SQL, step by step), `docs/middleware.html`, `docs/model-switching.html` and `docs/sql-errors.html` and `docs/sql-parser-rules.html`; `docs/report-queries.html` for why reports send their queries at once.